

Overview

Full i18n stack: AcceptHeaderLocaleResolver, 4-language message bundles, locale-aware currency/date formatting, TIMESTAMPTZ storage, multi-tenant locale context, and parameterised i18n tests.

Locale Resolution Chain

HTTP Request → Accept-Language: fr-FR

AcceptHeaderLocaleResolver → Locale.FRENCH

LocaleContextHolder.setLocale(locale)

TenantContextFilter enriches with tenantId, currency, zoneId

TenantAwareMessageSource → tenant bundle → fallback to mes

Message Bundle Files

File	Locale	Example Key
messages.properties	en (default)	order.placed=Your order {0} has been placed.
messages_fr.properties	French	order.placed=Votre commande {0} a été passée.
messages_de.properties	German	order.placed=Ihre Bestellung {0} wurde aufgegeben.
messages_ar.properties	Arabic (RTL)	order.placed=■ ■ ■ ■ ■ ■ ■ ■ {0}

Currency & Date Formatting

Locale	Currency Format	Date Format (MEDIUM)
en-US	\$1,234.56	Jun 17, 2026, 3:30:00 PM
de-DE	1.234,56 €	17.06.2026, 15:30:00
fr-FR	1 234,56 €	17 juin 2026 à 15:30:00
ar-SA	■ ■ ■ ■ ■ ■ ■ ■	■ ■ ■ ■ / ■ ■ ■ ■ / ■ ■ ■ ■

Timezone Handling Rules

- DB storage: TIMESTAMPTZ (PostgreSQL UTC). Always store as Instant.
- Customer.preferredZoneId stored as string, e.g. "Asia/Kolkata".
- OrderResponseMapper converts Instant → ZonedDateTime using customer zone.
- All byte[]↔String conversions use StandardCharsets.UTF_8 explicitly.

Testing i18n

Test Class	What It Verifies
I18nControllerTest	@CsvSource({en,fr,de}) → asserts response body language via Accept-Language header
CurrencyFormattingTest	\$1,234.56 for en-US vs 1.234,56 € for de-DE
TimezoneConversionTest	UTC stored time renders correctly in Europe/London and Asia/Kolkata